ОТЧЁТ
о выполнении практической работы № 7

Общие сведения

Образовательная организация: Институт перспективных технологий и индустриального программирования
Дисциплина: Технологии разработки приложений на базе фреймворков
Практическая работа: № 7. «Основы баз данных и SQL»
Группа: ЭФБО-08-24
Студент: Евтушенко Егор Максимович
Преподаватель: Ящун Т. В.
Вариант (тема): Платформа совместных покупок
Год выполнения: 2026

1. Цель работы

Изучить основные понятия реляционных баз данных и языка SQL, освоить работу с СУБД SQLite (проектирование таблиц, типы данных, первичные и внешние ключи, запросы на выборку, фильтрацию, сортировку, агрегирующие функции и группировку) и перенести индивидуальный проект с JSON-хранилища на реляционное хранение данных, получая результаты SQL-запросов в Django-представлениях.

2. Выполненные материалы Яндекс Практикума

Раздел 05 «Базы данных и Django ORM», тема «Введение в базы данных»: реляционная модель; SQL — язык структурированных запросов; первые запросы и первые результаты; работа с базой данных из Python; получение данных; практика запросов; сортировка, ограничение и сдвиг выборки; агрегирующие функции; группировка записей в выборке.

3. Доработка индивидуального проекта

Название: Платформа совместных покупок (Joint Purchases).
Проект получил слой реляционного хранения данных: спроектирована и создана база данных SQLite, данные перенесены из JSON, разработаны SQL-запросы, результаты которых получают Django-представления и отображают на веб-страницах.
Спроектированная структура базы данных (три таблицы):

- purchases (совместные покупки): id, product, price, min_participants, max_participants;
- users (пользователи): id, name, email;
- participations (участия): id, purchase_id, user_id, is_cancelled.

Поле id каждой таблицы — первичный ключ (PRIMARY KEY). Поля participations.purchase_id и participations.user_id — внешние ключи (FOREIGN KEY) на purchases.id и users.id.
Логическое значение is_cancelled хранится как целое число (0 — активно, 1 — отменено).
Организация файлов работы с базой данных:

- sql/schema.sql — SQL-команды создания таблиц и ключей;
- sql/queries.sql — проверенные SQL-запросы проекта;
- db/database.py — подключение к SQLite (get_connection(), PRAGMA foreign_keys = ON);
- db/import_json.py — перенос данных из JSON в таблицы (executemany, параметры ?, commit/rollback);
- data/joint_purchases.sqlite — файл базы данных.

Разработанные SQL-запросы (sql/queries.sql) — 11 запросов, не менее двух каждого вида:

- выборка списков: список покупок и список пользователей (SELECT ... ORDER BY);
- фильтрация: покупка по id и подбор покупок по бюджету (WHERE, параметр ?);
- соединение таблиц: список участий и одно участие (JOIN purchases и users);

• агрегатные функции: число активных участий (COUNT) и цены покупок (MIN, AVG, MAX);

• группировка: число участников по покупкам (GROUP BY, LEFT JOIN) и покупки, набравшие минимум участников (HAVING);

• дополнительно: покупки без активных участников (подзапрос NOT IN).

Получение данных из SQLite в Django-представлениях:

• соединение с базой через модуль sqlite3, conn.row_factory = sqlite3.Row — доступ к полям по именам;

• результат запроса передаётся в шаблон через контекст;

• значения от пользователя (бюджет) подставляются параметром ?, а не конкатенацией строк.

Пользовательские страницы, работающие с базой данных:

• /purchases/ — список покупок (выборка из SQLite);

• /purchases/<id>/ — отдельная покупка (параметризованный запрос, статус 404 для отсутствующей);

• /purchases/search/ — подбор покупок по бюджету (параметр из формы GET);

• /participations/ — список участий (соединение трёх таблиц);

• /participations/<id>/ — отдельное участие (JOIN, параметр);

• /participations/statistics/ — статистика участия (группировка и подсчёт).

Навигация сайта обеспечивает переходы между разделами «Главная», «Покупки», «Участия», «Статистика» через именованные URL Django; на странице покупок добавлена ссылка на подбор по бюджету.

Репозиторий: https://github.com/NE4EL/Joint-purchases

4. Результат работы

• создана база данных SQLite со структурой предметной области и внешними ключами;

• данные перенесены из JSON-файлов в таблицы скриптом импорта;

• разработаны и проверены 11 SQL-запросов в sql/queries.sql;

• Django-представления получают данные из SQLite и отображают их на веб-страницах;

• реализованы подбор покупок по бюджету и страница статистики;

• JSON-хранилище, классы моделей и консольная версия сохранены; Django ORM не вводился; README.md обновлён.

5. Проверка

```
python manage.py check   # без ошибок
pytest -v                # 20 тестов проходят
flake8 . --extend-exclude=venv   # без замечаний
```

• все страницы открываются (статус 200, для несуществующей покупки — 404);

• данные на страницах соответствуют содержимому таблиц SQLite (проверено запросами COUNT и выборками).

6. Вывод

В ходе выполнения ПР7 индивидуальный проект получил слой реляционного хранения данных. Спроектирована и создана база данных SQLite с первичными и внешними ключами, данные перенесены из JSON-файлов в таблицы, разработаны SQL-запросы (выборка, фильтрация, соединение таблиц, агрегатные функции, группировка и фильтрация групп, подзапрос), результаты которых получают Django-представления и отображают на веб-страницах. Значения пользователя передаются в запросы параметрами. Работоспособность предыдущих этапов сохранена. Цель практической работы достигнута.